

If threads are so important, how do programmers create and manage them easily in programs?

By: Bilal Ahmed

Thread Libraries

What is a Thread Library

A thread library provides the programmer with an API for creating and managing threads.

It allows developers to:

- Create threads
- Terminate threads
- Synchronize threads
- Manage thread communication

Thread libraries can be implemented in user space or kernel space.

Why Do We Need Thread Libraries?

- Simplify thread creation and management
- Improve program responsiveness
- Support concurrent execution
- Allow better utilization of multicore processors
- Reduce complexity for developers

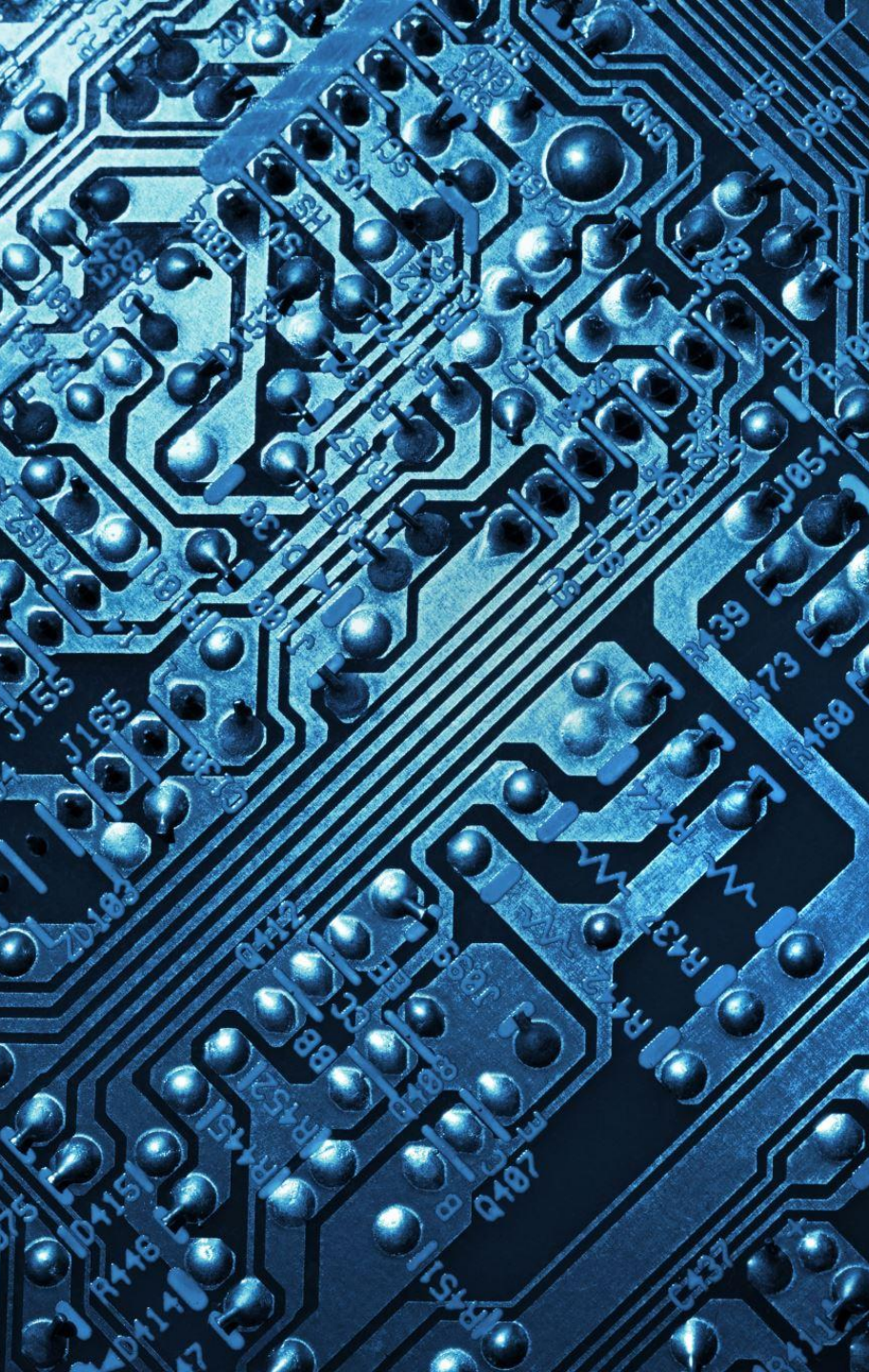
Ways to Implement Thread Libraries



USER-LEVEL THREAD
LIBRARY

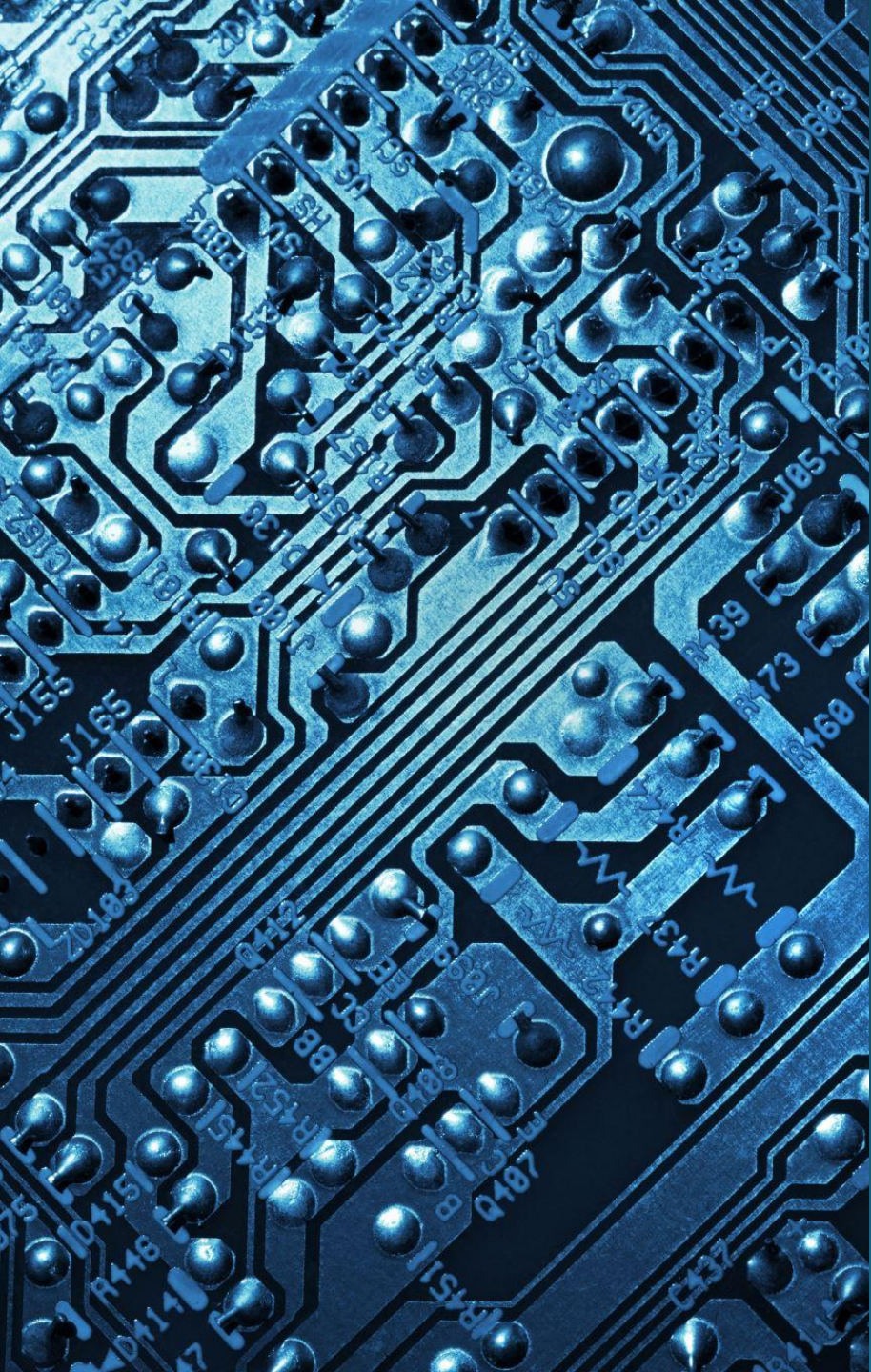


KERNEL-LEVEL
THREAD LIBRARY



User-Level Thread Library

- Implemented completely in user space
- No direct support from the operating system kernel
- Code and data structures remain in user memory
- Function calls are local function calls
- Faster because no system call is required



Kernel-Level Thread Library

- Supported directly by the operating system
- Code and data structures exist in kernel space
- Function calls usually involve system calls
- Slightly slower because of kernel involvement
- Provides better integration with the operating system

Types of Thread Libraries

POSIX Pthreads

Win32 Threads

Java Threads

POSIX Pthreads

Pthreads stands for POSIX Threads and can be implemented as user-level and kernel-level.

It is a standard API for thread creation and synchronization.

Commonly used in UNIX and Linux systems.

Provides functions such
as:

```
pthread_create()  
pthread_exit()  
pthread_join()  
pthread_mutex_lock()
```

Sample Code

```
#include <stdio.h>
#include <pthread.h>

void* task(void* arg) {
    printf("Pthread is running\n");
    return NULL;
}

int main() {
    pthread_t thread;

    pthread_create(&thread, NULL, task, NULL);
    pthread_join(thread, NULL);

    return 0;
}
```

Win32 Threads

Thread library provided by Microsoft Windows (only kernel-level).

Used for thread management in Windows-based applications.

Common functions include:

- CreateThread()
- ExitThread()
- WaitForSingleObject()
- CloseHandle()

Sample Code

```
#include <windows.h>
#include <stdio.h>

DWORD WINAPI task(LPVOID arg) {
    printf("Windows thread is running\n");
    return 0;
}

int main() {
    HANDLE thread;

    thread = CreateThread(NULL, 0, task, NULL, 0, NULL);
    WaitForSingleObject(thread, INFINITE);

    return 0;
}
```

Java Thread Library

- Java provides its own built-in thread API.
- Threads can be created and managed directly in Java programs.
- Java threads make multithreading easier and more portable across platforms.



Important Thread Methods in Java

`start()` → starts a new thread

`run()` → contains the code executed by the thread

`sleep()` → pauses a thread for some time

`join()` → waits for another thread to finish

`isAlive()` → checks if a thread is still running

Sample Code

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Java thread is running");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
        t1.start();  
    }  
}
```

Comparison of Thread Libraries

Feature	Pthreads	Win32 Threads	Java Threads
Platform	UNIX/Linux	Windows	Cross-platf orm
Ease of Use	Moderate	Moderate	Easy
Portability	High	Low	Very High
Language Support	C/C++	C/C++	Java

Python Threads

Python threads are **kernel-level threads** (mapped to native OS threads).

On Linux/UNIX → **POSIX threads (Pthreads)**
On Windows → **Win32 threads**

Managed by the **threading module** in Python.

Python Thread Applications

Task	Thread Role	Benefit
Data Loading & Preprocessing	Each thread loads and pre-processes different data chunks	Speeds up I/O, memory operations, and preparation time
Model Inference (I/O-heavy, e.g., network calls)	Threads handle multiple input requests	Reduces latency in real-time prediction
Web Scraping	Each thread fetches data from a different URL	Concurrent downloads, faster data collection
Monitoring & Logging	Threads run in the background while main model training happens	Keeps track of training, logs, and performance in real-time

Important Limitation

Python threads **do not run CPU-bound tasks in true parallel** because of the GIL.

For **CPU-heavy AI tasks** (like training a neural network), **multiprocessing** or GPU parallelism is better.

Sample code without thread

```
import time

def task(name):
    for i in range(3):
        print(f"{name} is running iteration {i+1}")
        time.sleep(1) # Simulate work

def main():
    task("Task 1")
    task("Task 2")

if __name__ == "__main__":
    main()
```

```
Task 1 is running iteration 1
Task 1 is running iteration 2
Task 1 is running iteration 3
Task 2 is running iteration 1
Task 2 is running iteration 2
Task 2 is running iteration 3
```

Sample code with thread

Task 1 is running iteration 1
Task 2 is running iteration 1
Task 1 is running iteration 2
Task 2 is running iteration 2
Task 1 is running iteration 3
Task 2 is running iteration 3

```
import threading
import time

def task(name):
    for i in range(3):
        print(f"{name} is running iteration {i+1}")
        time.sleep(1) # Simulate work

def main():
    t1 = threading.Thread(target=task, args=("Task 1",))
    t2 = threading.Thread(target=task, args=("Task 2",))

    t1.start()
    t2.start()

    t1.join()
    t2.join()

if __name__ == "__main__":
    main()
```

Applications of Thread Libraries

Web servers handling multiple users

Operating systems

Games and multimedia applications

Robotics and embedded systems

Banking and transaction systems

Real-time monitoring systems

Challenges of Using Thread Libraries

Race conditions

Deadlocks

Synchronization issues

Debugging complexity

Resource sharing problems

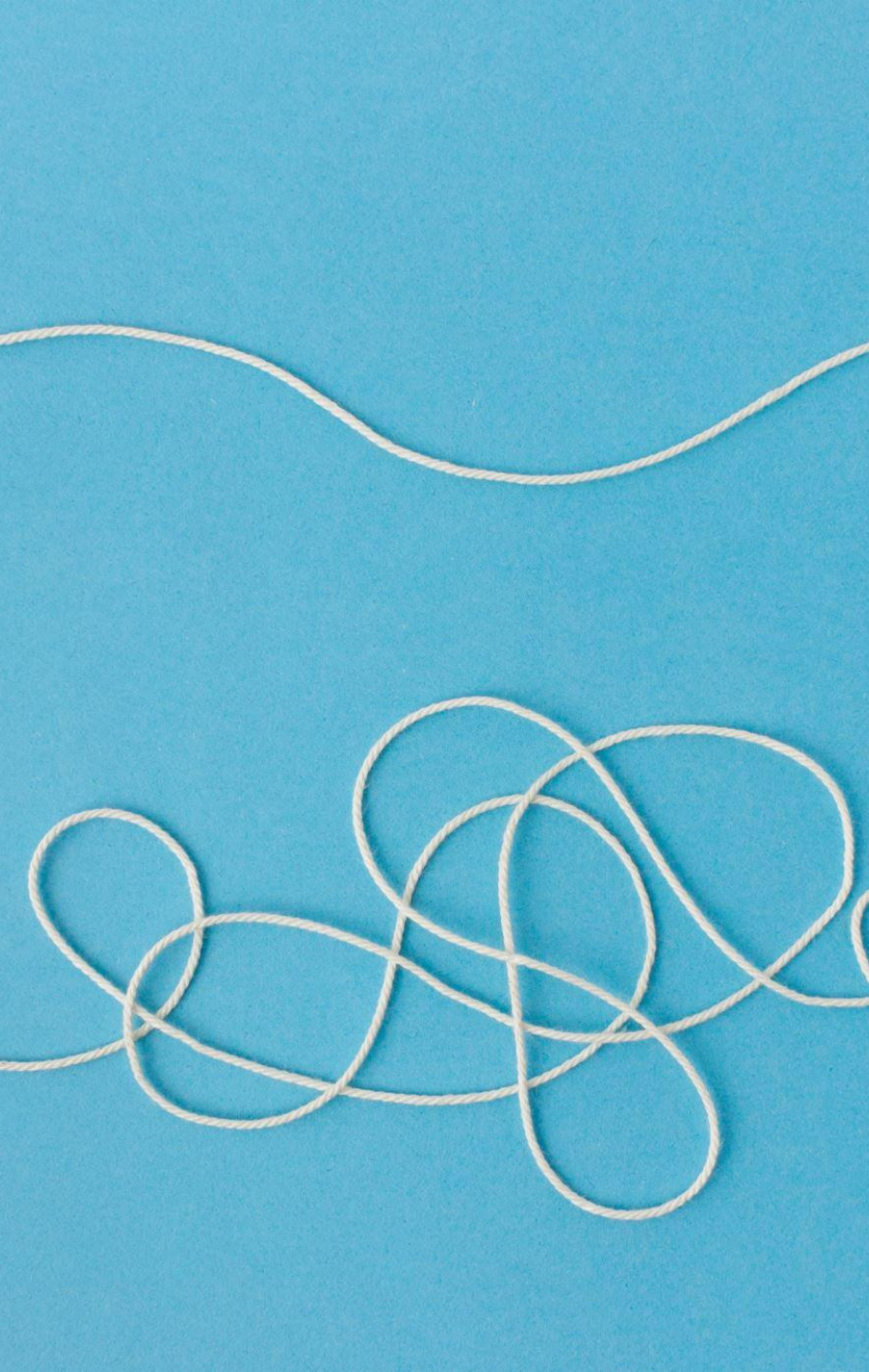
A Race...

Imagine a ticket booking website:

- Two users try to book the **last available seat** at the exact same time.
- Both see the seat as available and proceed to book it.

Question:

- What could go wrong if the system does not control access to shared data?
- How can we ensure that **only one booking succeeds?**



Solution

- By using **process/thread synchronization mechanisms** to prevent race conditions.

Threads in a System

Modern systems may run **hundreds or thousands of threads** concurrently or in parallel.

Threads often **share user data**.

The operating system continuously updates **data structures** to manage multiple threads.

Key Issue: Improper access to shared data can lead to **race conditions**.

Race Condition

Occurs when multiple threads/processes access shared data **without proper control**.

Result: Possible corruption of data values or inconsistent system state.

Example: Two threads incrementing the same counter at the same time may produce incorrect results.

Race Condition Counter Example

- **Problem:** Two processes manipulate a shared variable `count` concurrently.
- **Observation:** Even though high-level statements seem atomic (in one go), they are executed as **multiple low-level steps**.

- `count++` may be broken down as:

```
register1 = count  
register1 = register1 + 1  
count = register1
```

- `count--` may be broken down as:

```
register2 = count  
register2 = register2 - 1  
count = register2
```

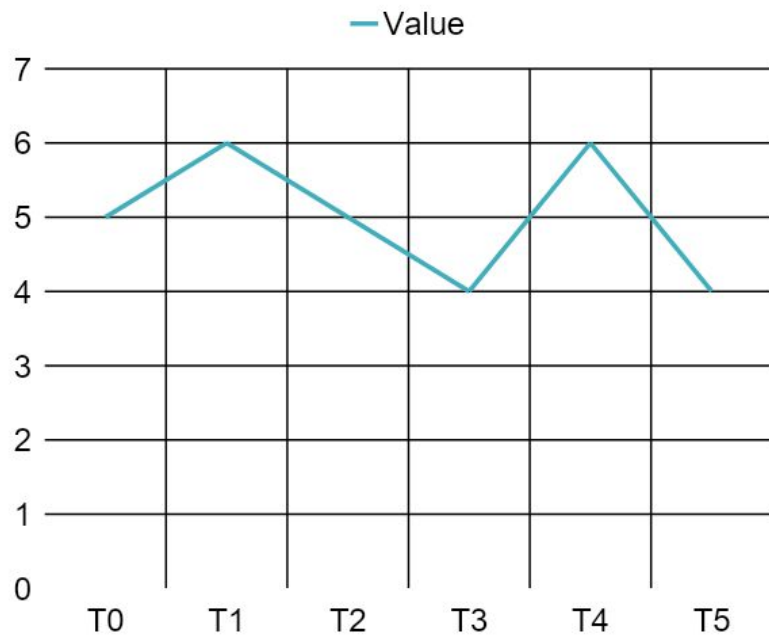
Atomic

- `count++` looks like **one single action**, but the computer does not execute it in one step.
- Instead, the CPU breaks it into smaller steps like:
 - Read the value of `count` from memory
 - Store it in a register (temporary CPU storage)
 - Add 1 to it
 - Write the updated value back to memory
- **Why this matters:**
 - If two threads execute at the same time, they may:
 - Read the same old value
 - Modify it separately
 - Overwrite each other's updates
- That's why race conditions happen.

Interleaving of Steps

•**Result:** count = 4 (incorrect) → Race condition!

Operation and Register Values Over Time

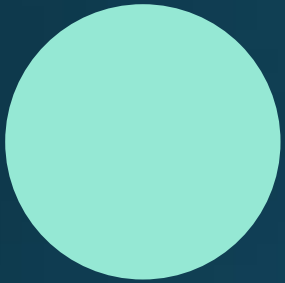


T ₀	producer: register1 = count	register1	5
T ₂	consumer: register2 = count	register2	5
T ₄	producer: count = register1	count	6

Where can race conditions
occur in a program? OR
What that section should
be called?

Critical Section Problem

What is a Critical Section?



A critical section is a part of a program where shared resources are accessed.



Only one process or thread should execute inside the critical section at a time.



Example: `balance = balance + 100;`



If two threads update the same balance together, the final value may become incorrect.

Background

- In multitasking systems, multiple processes or threads may execute at the same time.
- These processes often share common resources such as:
 - Variables
 - Files
 - Printers
 - Databases
 - Memory locations
- When multiple processes try to access the same shared resource simultaneously, problems may occur.

Critical-Section Problem

```
while (true) {  
    entry section  
        critical section  
    exit section  
        remainder section  
}
```

The **critical-section problem** is about designing a protocol to **safely share data** among processes.

Processes must **synchronize their activities** to avoid conflicts.

Each process has the following structure:

- **Entry Section:** Requests permission to enter the critical section
- **Critical Section:** Accesses shared data/resources
- **Exit Section:** Releases the resource and allows others to enter
- **Remainder Section:** Executes the rest of the program

Goal: Ensure **safe and orderly access** to shared resources.

Requirements of Critical Section Solution

Mutual Exclusion: Only one process can execute in the critical section at a time.

Progress: If no process is in the critical section, one of the waiting processes must be selected **without indefinite delay**.

Bounded Waiting: A process should not wait forever; there is a **limit on waiting time**.

No Assumptions in Concurrency

In concurrent systems, we **cannot assume** anything about:

- The **speed of processes**
- The **order of execution**

Multiple processes may run **simultaneously or unpredictably**.

Example – Open File List

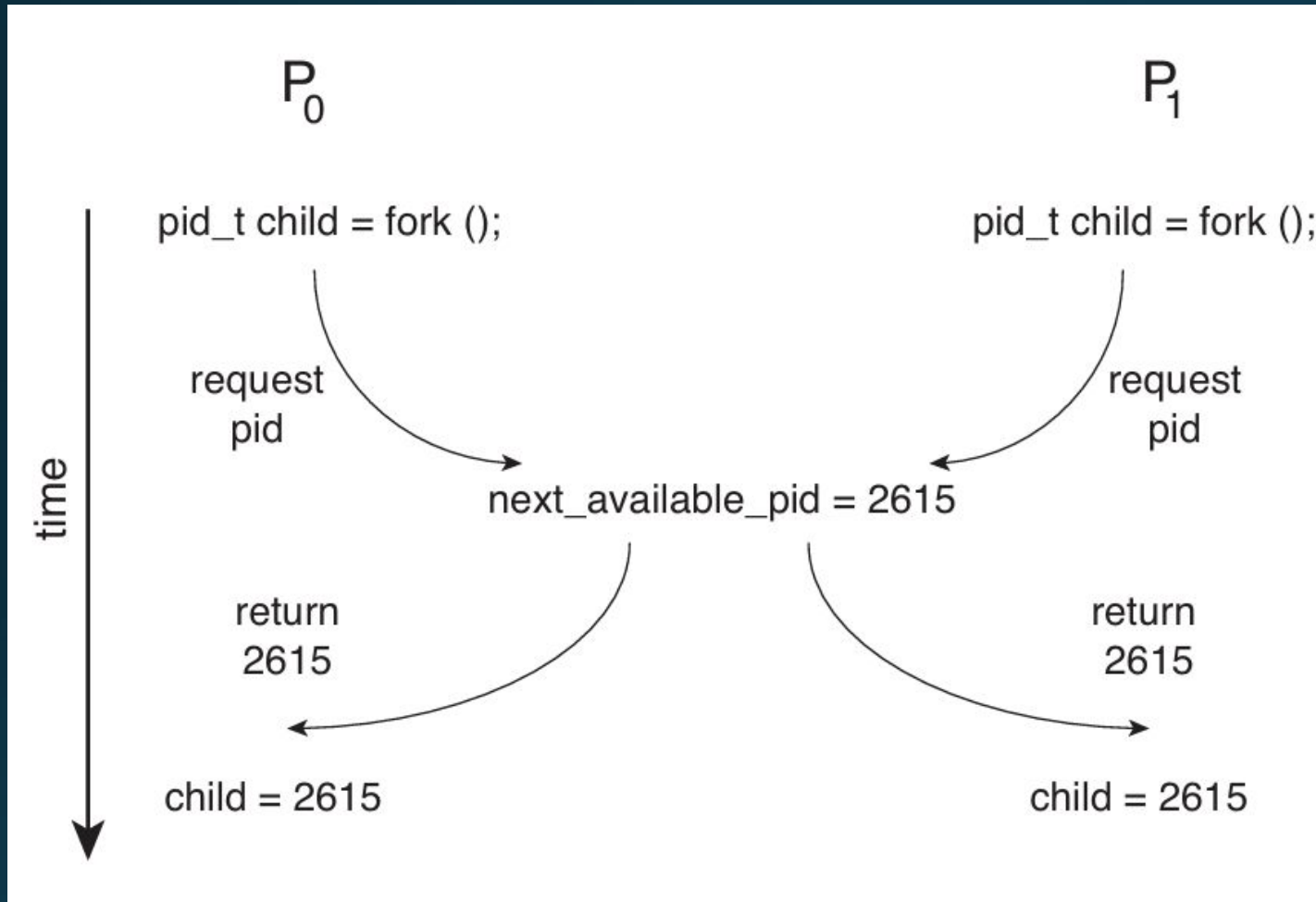
OS maintains a **list of all open files**.

When a file is
opened/closed:

It is
added/removed
from the list.

If two processes
update the list
simultaneously:

The list may
become
inconsistent or
corrupted.



Example – Process ID (PID) Allocation

When a process creates a child using `fork()`:

- A new **process ID (PID)** is assigned.

OS uses a variable like **next available PID**.

If two processes call `fork()` at the same time:

- Both may receive the **same PID** → **Error**

Java Example

```
class BankAccount {
    int balance = 0;

    public void deposit() {
        for (int i = 0; i < 100000; i++) {
            balance++; // NOT thread-safe
        }
    }
}

public class RaceConditionDemo {
    public static void main(String[] args) throws InterruptedException {

        BankAccount account = new BankAccount();

        Thread t1 = new Thread(() -> account.deposit());
        Thread t2 = new Thread(() -> account.deposit());

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Balance: " + account.balance);
    }
}
```

Why incorrect?

`balance++;` is the problem.

It is NOT atomic. Internally it does:

- Read balance
- Add 1
- Write back

If two threads do this at the same time:

They may overwrite each other's updates

Responsibility of OS Developers

Kernel developers must ensure:

- **Mutual exclusion** when accessing shared data
- Proper **synchronization mechanisms**

Goal: Make the operating system
free from race conditions

If OS developers
must ensure
mutual exclusion,
how can the
system itself help
enforce it?

Disabling Interrupts – Hardware solution

In a **single-core system**, we can solve the critical-section problem by:

- Disabling interrupts while updating shared data

This ensures:

- No interruption during execution
- Instructions run **completely and in order**
- No other process can modify the shared variable

Why It Works

Prevents **context switching**
(no preemption)

Guarantees **exclusive**
access to the critical section

Simple and effective for
single-core systems

Limitations in Multiprocessor Systems

Not practical in
multi-core/multiprocessor systems

Disabling interrupts
requires:

Sending signals to
all processors

This causes:

Delays in entering
critical sections
Reduced **system
efficiency**

Software Solution

A classic software-based solution
to the critical-section problem

Does not rely on special hardware
instructions

Uses simple
variables and logic to
achieve:

Mutual Exclusion
Progress
Bounded Waiting

Peterson's Solution

Designed for **two processes** (P₀ and P₁)

Uses two shared variables:

- `flag[i]` → indicates if process *i* wants to enter critical section
- `turn` → indicates whose turn it is

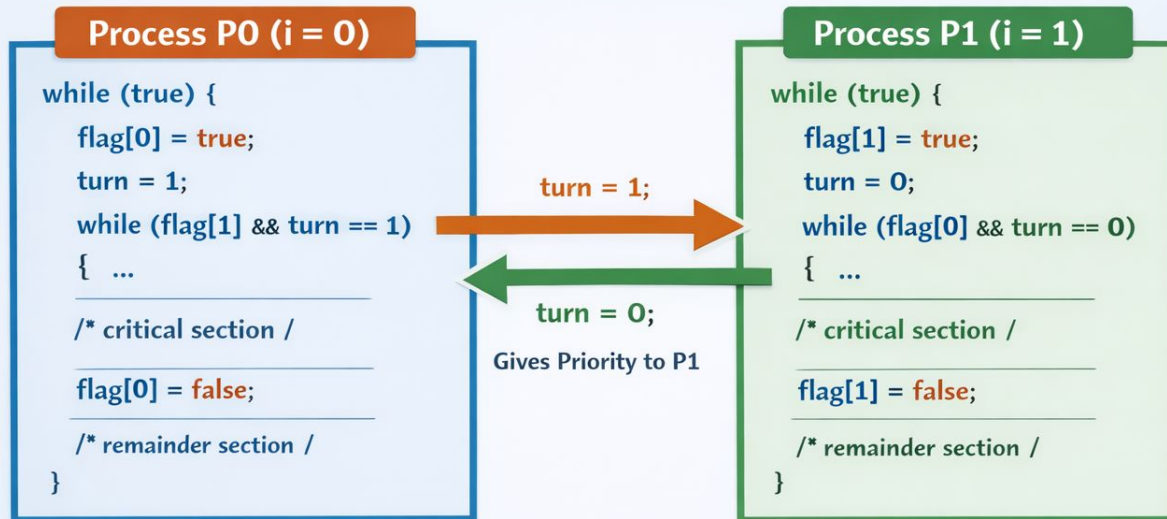
Idea:

- A process **signals its intention** to enter the critical section
- Then **gives priority to the other process**

Peterson's Solution (Pseudocode)

- For Process P_i ($i = 0$ or 1 , $j =$ other process)
 - Pure **software-based solution**
 - Elegant but mainly **theoretical (educational use)**
 - Helps understand **how synchronization works without hardware support**
-

Peterson's Solution



 `flag[i]` = "I want to enter"

 `turn` = "Who goes first?"

Illustration

- Assume ; after second while.
- { is a mistake, ignore it.

Practical Implications of Peterson's Solution

Modern CPUs may use **instruction reordering (optimization)**

Not reliable on **multicore or modern architectures** without guarantees

Mainly used for **conceptual understanding of synchronization**

Not used in real-world operating systems

Since pure software solutions have limitations, modern systems rely on **hardware-assisted synchronization**.

Moving Toward Practical Hardware Support

Thread 1:

```
data = 100  
MEMORY_BARRIER  
flag = true
```

Thread 2:

```
while (flag == false)  
    ; // wait
```

```
MEMORY_BARRIER  
print(data)
```

Memory Barriers

Ensure **ordering of memory operations**

Prevent CPU/compiler from **reordering instructions**

Used to maintain **consistency in multithreaded execution.**

Problem: Low-level and difficult to use correctly in large programs

Hardware Instructions

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = false;  
  
    /* remainder section */  
} while (true);
```

Provide **atomic (indivisible)** operations

Examples: Test-and-Set,
Compare-and-Swap (CAS)

Used to implement **locks and synchronization primitives**

Problem: Complex to program and may reduce performance due to busy waiting

Atomic Variables

Variables with **atomic operations**
(**single-step execution**)

No intermediate state visible to other threads

Used for **counters and lock-free operations**

Problem: Limited to simple operations; not suitable for complex synchronization

So, these hardware methods also have limitations as they are implemented at low level?
What should we do?

Mutex Lock

Introduction to Mutex Lock

Hardware-based solutions are **complex and hard to use for programmers**

Operating systems provide **higher-level tools** for synchronization

The simplest tool is the **mutex lock**

“Mutex” stands for **Mutual Exclusion**

Used to **protect critical sections** and prevent race conditions

How Mutex Works

- A process must **acquire the lock** before **entering** the critical section
- It must **release the lock** after **exiting** the critical section
- **Functions:**
 - `acquire()` → locks the critical section
 - `release()` → unlocks the critical section
- **Key Idea:**
 - Only **one thread/process** can access **the critical section** at a time
 - Ensures **safe and consistent access** to shared data

`acquire()`

```
<> C

acquire() {
    while (available == false)
        ; // busy wait

    available = false;
}
```

`release()`

```
<> C

release() {
    available = true;
}
```

How Mutex Works

- A mutex lock uses a **boolean variable available** to indicate lock status
- If available = true:
 - Lock is available
 - acquire() succeeds and lock becomes **unavailable**
- If available = false:
 - Lock is already in use
 - Process is **blocked until the lock is released**

```
while (true) {
```

```
    acquire lock
```

```
    critical section
```

```
    release lock
```

```
    remainder section
```

```
}
```

Getting Back to Java Example

```
class BankAccount {
    int balance = 0;

    public void deposit() {
        for (int i = 0; i < 100000; i++) {
            balance++; // NOT thread-safe
        }
    }
}

public class RaceConditionDemo {
    public static void main(String[] args) throws InterruptedException {

        BankAccount account = new BankAccount();

        Thread t1 = new Thread(() -> account.deposit());
        Thread t2 = new Thread(() -> account.deposit());

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Balance: " + account.balance);
    }
}
```

Solution

ReentrantLock is a **Java** implementation of a mutex-like lock

```
import java.util.concurrent.locks.ReentrantLock;

class BankAccount {
    int balance = 0;
    ReentrantLock lock = new ReentrantLock();

    public void deposit() {
        for (int i = 0; i < 100000; i++) {
            lock.lock();    // acquire mutex
            balance++;      // critical section
            lock.unlock();  // release mutex
        }
    }
}

public class RaceConditionDemo {
    public static void main(String[] args) throws InterruptedException {

        BankAccount account = new BankAccount();
        //Thread 1
        Thread t1 = new Thread(() -> account.deposit());
        //Thread 2
        Thread t2 = new Thread(() -> account.deposit());
        // Start threads
        t1.start();
        t2.start();
        //Wait for completion
        t1.join();
        t2.join();
        // Final result
        System.out.println("Final Balance: " + account.balance);
    }
}
```

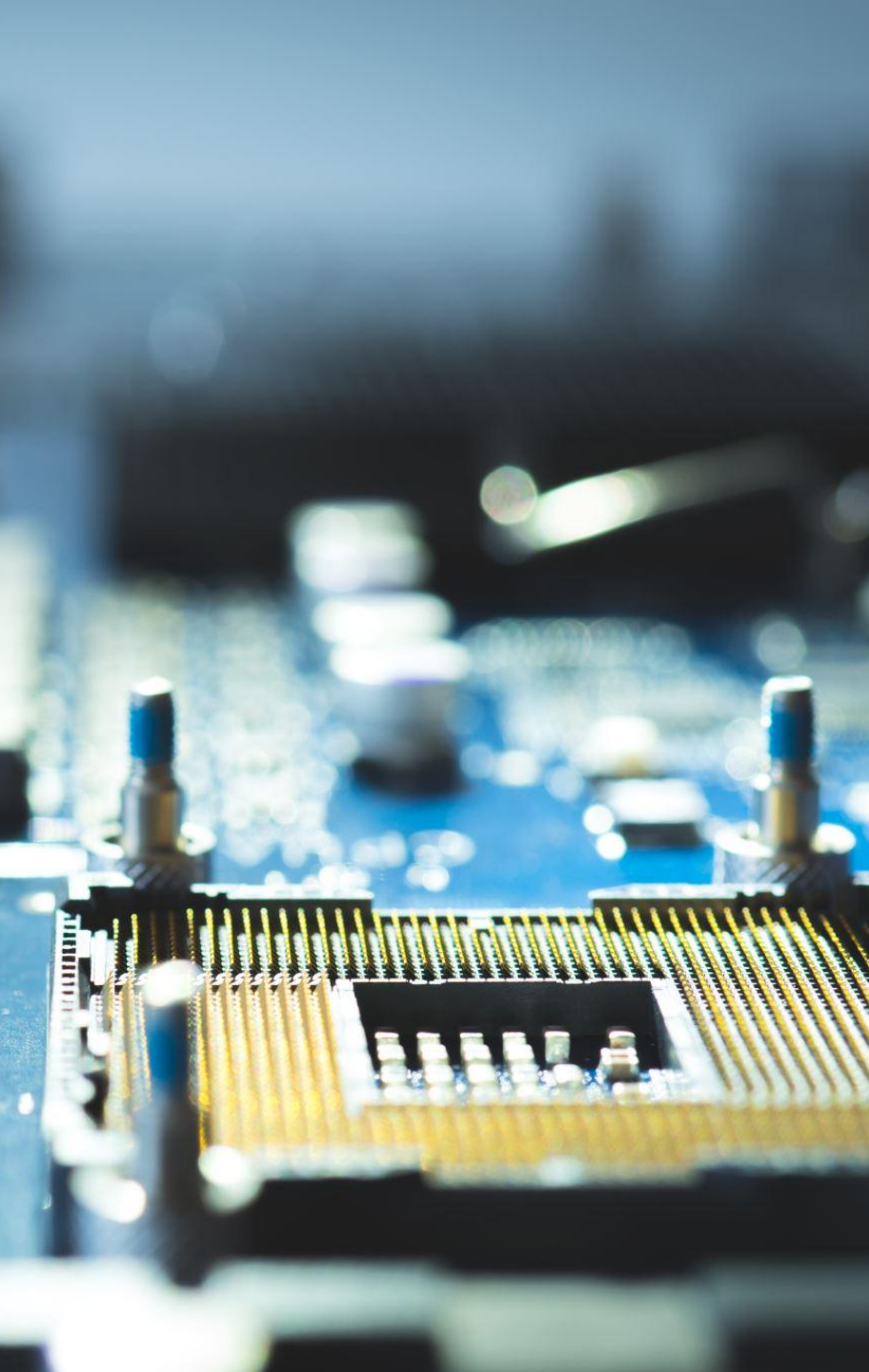
Atomicity Requirement in Mutex

The operations **acquire()** and **release()** must be **atomic**, meaning they execute as a **single, uninterrupted step**

If not atomic, multiple processes may read or modify the lock at the same time

This can lead to **race conditions on the lock itself** and break mutual exclusion

Without atomicity, two processes may enter the critical section simultaneously or corrupt the lock state



Busy Waiting (Spinlock Issue)

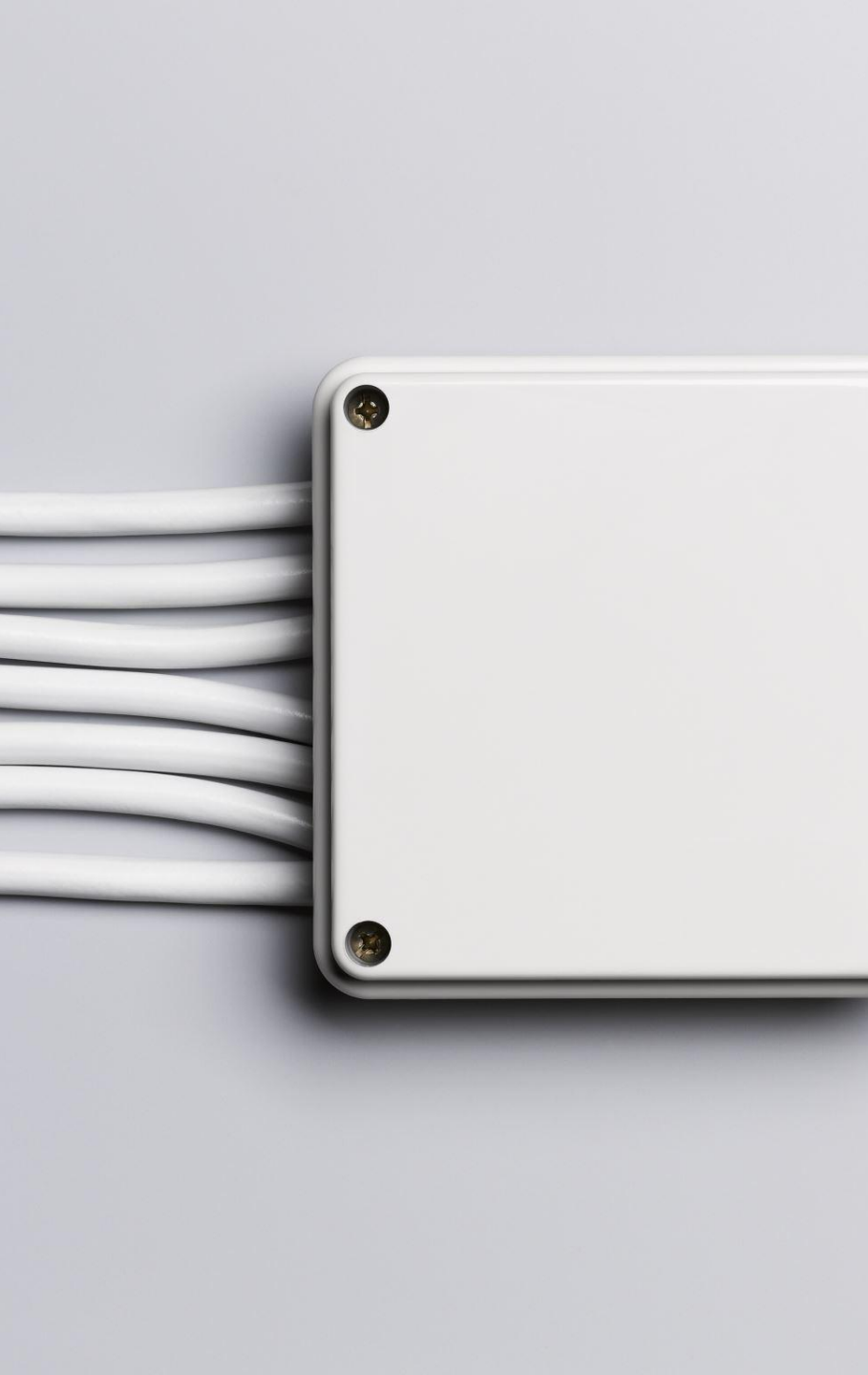
- Main disadvantage of this mutex implementation is **busy waiting**
- A process continuously loops in `acquire()` while waiting for the lock
- This wastes **CPU cycles** that could be used by other processes

Spinlock Concept

This type of mutex is called a **spinlock**

Process “spins” (keeps checking) until lock becomes available

Common issue in systems with **single CPU core**



Advantages of Spinlocks

- No **context switch overhead** while waiting
- Can be efficient in **multicore systems**
- Useful when critical section is **very short**
- One thread can spin while another runs on a different core

What if we don't
want only ONE
thread...

what if we want
LIMITED threads
(e.g., 3) to enter at
the same time?

Mutex limitation

"Mutex is too strict —
it allows only 1.

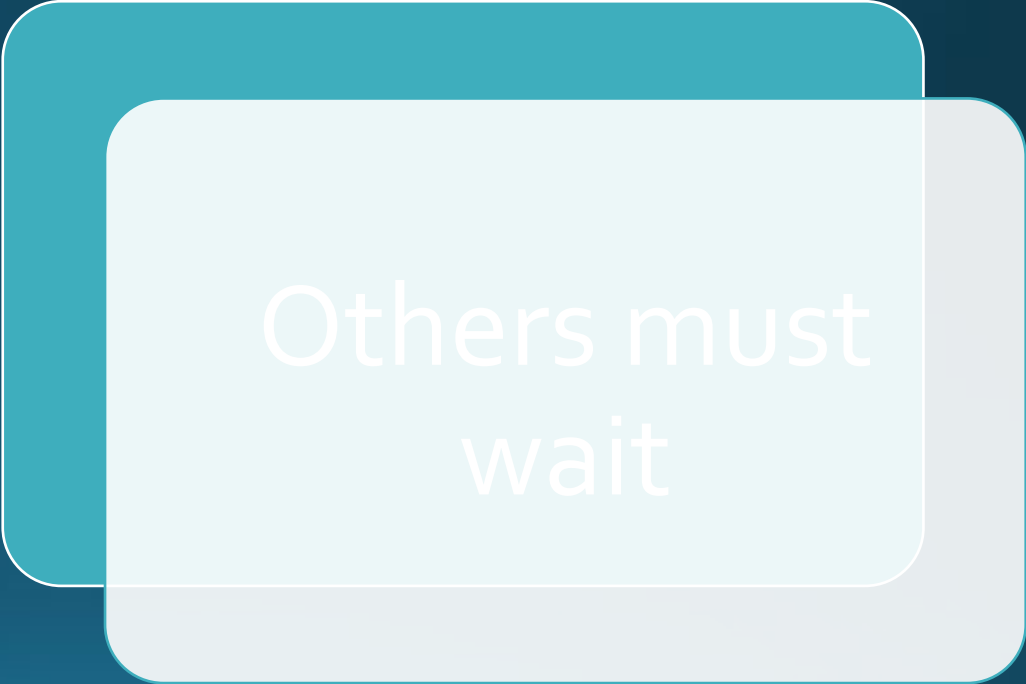
But sometimes, we
want **controlled
sharing**, not complete
restriction."

Real-life example



Only 3 students
allowed in lab
at a time

The diagram consists of three overlapping rounded rectangles. The top rectangle is teal, the middle one is light blue, and the bottom one is light gray. The text is centered in the middle rectangle.



Others must
wait

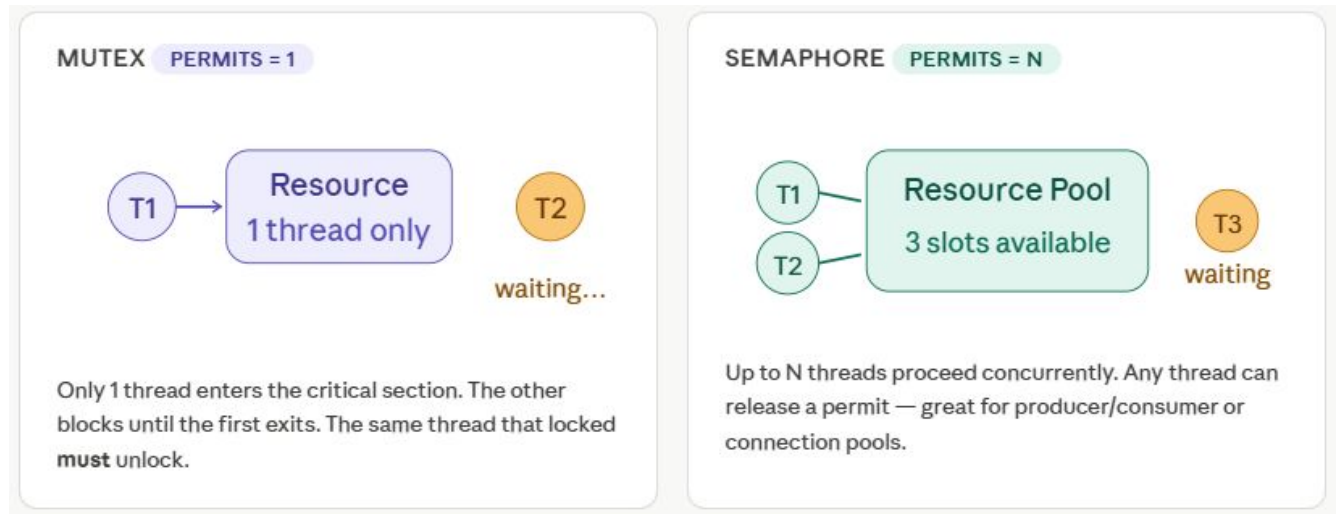
The diagram consists of three overlapping rounded rectangles. The top rectangle is teal, the middle one is light blue, and the bottom one is light gray. The text is centered in the middle rectangle.



Semaphore

Illustration

- **Mutex** = exclusive lock for **1 thread only** (binary: locked/unlocked). The thread that locks it must unlock it. Think of it as a single-person restroom key.
- **Semaphore** = a counter that **allows N threads concurrently**. Any thread can release it. Think of it as a parking lot with N spots.



Mutex vs Semaphore

Mutex = one key 

Semaphore = multiple keys   

Mutex → only one person can enter

Semaphore → limited number can enter

Connection pool with limited DB connections (say 3), and multiple threads compete for them.

A semaphore is the right tool here; a mutex would only allow 1 thread at a time (wasteful).

✗ Wrong use — S=3

One shared balance variable. Semaphore(3) lets 3 threads read-modify-write simultaneously → race condition. The resource is **not splittable**.



✓ Right use — S=3

3 independent DB connections. Each thread gets its **own** connection object — no sharing inside the critical section. Resources are identical but separate.



Semaphore is not for single resource

Semaphore

- Semaphore is just a **generalized mutex**
- **Mutex**: 1 Thread
- **Semaphore**: N Thread



Semaphore

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

```
signal(S) {  
    S++;  
}
```

- Semaphore is just a counter with two operations
 - wait(S) // decrease
 - signal(S) // increase
- S = number of available slots (keys)

Counting vs Binary Semaphore

Counting Semaphore

- Value range: $0 \rightarrow \infty$ (any non-negative number)
- Allows **multiple processes/threads**
- Used for **resource management** (e.g., limited seats, printers).

Binary Semaphore

- Value range: **0 or 1**
- Allows **only one process/thread at a time**
- Behaves like a **mutex (mutual exclusion)**

Counting Semaphore for Resource Management

Used to control access to **limited resources**

Initialized with total number of resources

How it Works

S = number of available resources

When a process wants a resource:

- Calls `wait(S) → S--`
- If $S > 0 \rightarrow$ enters
- If $S = 0 \rightarrow$ **waits (blocked)**

When a process releases a resource:

- Calls `signal(S) → S++`

How it Works

Key Behavior

- $S > 0 \rightarrow$ resources available
- $S = 0 \rightarrow$ all resources in use
- New processes must **wait**

Example

- 3 printers $\rightarrow S = 3$
- 3 processes use them $\rightarrow S = 0$
- 4th process $\rightarrow$ **waits**
- One finishes $\rightarrow S = 1 \rightarrow$ next process enters

Java Binary Semaphore

```
import java.util.concurrent.Semaphore;

class BankAccount {
    int balance = 0;

    // Binary semaphore (acts like mutex)
    Semaphore sem = new Semaphore(1);

    public void deposit() {
        for (int i = 0; i < 100000; i++) {
            try {
                sem.acquire(); // enter critical section
                balance++;     // critical section
            } catch (InterruptedException e) {
                e.printStackTrace();
            } finally {
                sem.release(); // exit critical section
            }
        }
    }
}

public class RaceConditionDemo {

    public static void main(String[] args) throws InterruptedException {

        BankAccount account = new BankAccount();

        Thread t1 = new Thread(() -> account.deposit());
        Thread t2 = new Thread(() -> account.deposit());

        t1.start(); t2.start();
        t1.join();  t2.join();

        System.out.println("Final Balance: " + account.balance);
    }
}
```

Semaphore Real-life Example - connection pool with 3 DB and 6 clients

```
import java.util.concurrent.Semaphore;

class ConnectionPool {
    // Only 3 threads can hold a connection at once
    private final Semaphore semaphore = new Semaphore(3);

    public void queryDatabase(String threadName) throws InterruptedException {
        System.out.println(threadName + " waiting for connection...");
        semaphore.acquire();    // blocks if 0 permits left

        try {
            System.out.println(threadName + " got connection, querying...");
            Thread.sleep(2000); // simulate DB work
        } finally {
            semaphore.release(); // any thread can release!
            System.out.println(threadName + " released connection");
        }
    }
}

public class Main {

    public static void main(String[] args) {
        ConnectionPool pool = new ConnectionPool();

        // Launch 6 threads competing for 3 connections
        for (int i = 1; i <= 6; i++) {
            final String name = "Thread-" + i;

            new Thread(() -> {
                try {
                    pool.queryDatabase(name);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }).start();
        }
    }
}
```

Summary



Use cases – Mutex vs Semaphore

Scenario	Use	Why
Shared counter / bank balance	Mutex / $S = 1$	One resource, must serialize all access
DB connection pool (N connections)	Semaphore (N)	N independent resources, each thread works on its own
Rate limiting (max N req/sec)	Semaphore (N)	Allow N concurrent callers, throttle the rest
Producer-consumer signaling	Semaphore (o)	Start at 0, producer releases, consumer acquires
File write (one writer at a time)	Mutex / $S = 1$	Exclusive access required — one shared resource
Thread pool — max N workers active	Semaphore (N)	Limit concurrency without exclusive locking